

Rapport de TD n°2 : Programmation Impérative

DERREZ Lorenzo

September 18, 2026

1 ERRATUM

1. Il y a certains main où il manque des `return EXIT_SUCCESS` en fin de fonction.

Exercice 1 :

Opérations avec *argc* et *argv*

```
1 // Q1
2
3 int main(int argc, char* argv[]){
4     int i = 1;
5     while(i<argc){
6         printf("%s_\n", argv[i]);
7         i+=1;
8     }
9     return EXIT_SUCCESS;
10 }
11
12 // Q2
13
14 int calc_taille1(int argc, char* argv[]){
15     if(argc>1){
16         int taille = 0;
17         int i = 0;
18         while(argv[1][i] != 0){
19             taille +=1;
20             i +=1;
21         }
22         return taille;
23     }
24     else{
25         return 0;
26     }
27 }
28
29
30 int calc_taille2(int argc, char* argv[]){
31     if(argc>1){
32         return strlen(argv[1]);
33     }
34     else{
35         return 0;
36     }
37 }
38
```

```

39 // Q3
40 int lengthcompare(int argc, char* argv[]){
41     if(argc>2){
42         int taille1 = strlen(argv[1]);
43         int taille2 = strlen(argv[2]);
44
45         if(taille1>taille2){return taille1;}
46         else{return taille2;}
47     }
48     else{
49         if(argc==2){
50             return strlen(argv[1]);
51         }
52         else{return 0;}
53     }
54 }
55
56 //Q4
57 int est_voyelle(char caractere){
58     if (caractere == 'a' || caractere == 'e' ||
59         caractere == 'i' || caractere == 'o' ||
60         caractere == 'u' || caractere == 'y' ||
61         caractere == 'A' || caractere == 'E' ||
62         caractere == 'I' || caractere == 'O' ||
63         caractere == 'U' || caractere == 'Y')
64     {
65         return 1;
66     }
67     return 0;
68 }
69
70 int nb_voy(char *chaine){
71     int n = strlen(chaine);
72     int nb = 0;
73     int i = 0;
74
75     while (i < n)
76     {
77         if (est_voyelle(chaine[i]) == 1)
78         {
79             nb++;
80         }
81         i++;
82     }
83     return nb;
84 }
85
86
87 int main(int argc, char* argv[]){
88     int test1 = calc_taille2(argc, argv);
89     printf("%d\n", test1);
90     return 0;
91 }
92
93 // Test Q3 :
94 int main(int argc, char* argv[]){
95     int i = 1;
96     while(i<argc){
97         int test = nb_voy(argv[i]);

```

```

98         printf("%d\n", test);
99         i+=1;
100     }
101     return EXIT_SUCCESS;
102 }

```

Question 2 :

Implémentation des fonctions *palindrome*, *isalpha*, *tolower*

```

1
2 // Q1 :
3
4 int est_palindrome(char* chaine){
5     int n = strlen(chaine);
6     int i = 0;
7     int parcours = n/2;
8
9     int cpt = 0;
10
11     while(i<parcours && (chaine[i] == chaine[n-1-i])){
12         i+=1;
13         cpt+=1;
14     }
15
16     if(cpt == parcours){
17         return 1;
18     }
19     return 0;
20 }
21
22 // Test Q1
23
24 int main(int argc, char** argv){
25     int i = 1;
26     while(i < argc){
27         int test_i = est_palindrome(argv[i]);
28         if (test_i == 1){
29             printf("%s est un palindrome.\n", argv[i]);
30         }
31         else{
32             printf("%s n'est pas un palindrome.\n", argv[i]);
33         }
34         i+=1;
35     }
36     return EXIT_SUCCESS;
37 }
38
39 // Q2 :
40 // #include <ctype.h>
41
42 // int is_in_alph(char caractere)
43 // {
44 //     return isalpha(caractere);
45 // }
46
47 int is_in_alph(char caractere)

```

```

48 {
49     if ((caractere >= 'a' && caractere <= 'z') ||
50         (caractere >= 'A' && caractere <= 'Z'))
51     {
52         return 1;
53     }
54     else
55     {
56         return 0;
57     }
58 }
59
60 // Test Q2 :
61 int main(int argc, char** argv){
62     int i = 0;
63     while(i<argc){
64         int n_i = strlen(argv[i]);
65         int j = 0;
66         while(j<n_i){
67             int test_i = is_in_alph(argv[i][j]);
68             if(test_i){
69                 printf("%c_est_dans_l'alphabet.\n", argv[i][j]);
70             }
71             else{
72                 printf("%c_n'est_pas_dans_l'alphabet.\n", argv[i][j]);
73             }
74             j+=1;
75         }
76         i+=1;
77     }
78     return EXIT_SUCCESS;
79 }
80
81 // Q3 :
82 char maj_to_min(char car)
83 {
84     if (car >= 'A' && car <= 'Z')
85     {
86         return car + ('a' - 'A');
87     }
88     else
89     {
90         return car;
91     }
92 }
93
94 int main(int argc, char **argv)
95 {
96     int i = 1;
97
98     while (i < argc)
99     {
100         int n_i = strlen(argv[i]);
101         int j = 0;
102
103         while (j < n_i)
104         {
105             char test_j = maj_to_min(argv[i][j]);
106

```

```

107         printf("%c", test_j);
108
109         j += 1;
110     }
111
112     printf("\n");
113     i += 1;
114 }
115
116 return 0;
117 }
118
119 //Q4 :
120
121 // Amelioration de la fonction palindrome :
122 int est_palindrome_update(char* chaine){
123     int n = strlen(chaine);
124     int g = 0;
125     int d = n-1;
126
127     while(g<d){
128         while(is_in_alph(chaine[g]) == 0 && g<d){
129             g+=1;
130         }
131         while(is_in_alph(chaine[d]) == 0 && g<d){
132             d-=1;
133         }
134         if(g>d){
135             break;
136         }
137         if(maj_to_min(chaine[g]) == maj_to_min(chaine[d])){
138             d-=1;
139             g+=1;
140         }
141         else{
142             return 0;
143         }
144     }
145     return 1;
146 }

```

Exercice 3 :

Implémentation de la fonction *chaine miroir*

```

1
2 //Q1 :
3
4 void chaine_miroir(char* chaine){
5     int n = strlen(chaine);
6     int i = 0;
7     while(i<n){
8         printf("%c", chaine[n-1-i]);
9         i+=1;
10    }
11    printf("\n");
12 }

```

```

13
14 // Autre methode Q1 :
15 char* chaine_miroir_update(char* chaine){
16     int n = strlen(chaine);
17     int i = 0;
18     char* tab = malloc((n+1)*sizeof(char));
19     tab[n] = 0;
20     while(i<n){
21         printf("%c", chaine[n-1-i]);
22         tab[i] = chaine[n-1-i];
23         i+=1;
24     }
25     printf("\n");
26     return tab;
27 }
28
29
30 int main(int argc, char** argv){
31     int i = 1;
32     while(i<argc){
33         chaine_miroir(argv[i]);
34         i+=1;
35     }
36     return EXIT_SUCCESS;
37 }
38
39 // Q2 :
40 // Etant donne que ma fonction de Q1 ne modifie jamais le tableau, je me suis
41 // contente de mettre const char* en modification
42 void chaine_miroir_2(const char* chaine){
43     int n = strlen(chaine);
44     int i = 0;
45     while(i<n){
46         printf("%c", chaine[n-1-i]);
47         i+=1;
48     }
49     printf("\n");
50 }
51
52 int main(int argc, char** argv){
53     int i = 1;
54     while(i<argc){
55         chaine_miroir(argv[i]);
56         i+=1;
57     }
58     return EXIT_SUCCESS;
59 }
60 //Avec l'autre methode :
61
62 // Autre methode Q1 :
63 char* chaine_miroir_2_update(const char* chaine){
64     int n = strlen(chaine);
65     int i = 0;
66     char* tab = malloc((n+1)*sizeof(char));
67     tab[n] = 0;
68     while(i<n){
69         printf("%c", chaine[n-1-i]);
70         tab[i] = chaine[n-1-i];

```

```

71         i+=1;
72     }
73     printf("\n");
74     return tab;
75 }

```

Exercice 4 :

Implémentation de la fonction *itoa*

```

1
2 // Q1 :
3
4 void int_to_a(int code){
5     int taille = 1;
6     int odg = 1;
7
8     while(code>=odg*10){
9         odg = odg*10;
10        taille+=1;
11    }
12
13    char chaine[taille+1];
14    chaine[taille] = 0;
15    taille--;
16
17    int i = 0;
18
19    while(odg>0){
20        char val = '0' + (code/odg);
21        chaine[i] = val;
22        code = code%odg;
23        odg=odg/10;
24        printf("%c", val);
25        i+=1;
26    }
27    printf("\n");
28 }
29
30 // Q2 :
31
32 // Extension avec les entiers signes :
33 void int_to_a_signed(int code){
34     if(code >= 0){
35         int_to_a(code);
36     }
37     else{
38         // Il faudrait seulement gerer le cas INT_MIN autrement car cette methode
39         // ne fonctionne pas pour lui
40         printf("-");
41         int_to_a(code-2*code);
42     }
43 }
44
45 // Q3 :
46 // Test des fonctions :

```

```

47 // Test Q1
48 int main(int argc, char** argv){
49     int i = 1;
50     while(i<argc){
51         int t_i = atoi(argv[i]);
52         int_to_a(t_i);
53         i+=1;
54     }
55     return EXIT_SUCCESS;
56 }
57
58 // Test Q2 :
59 // A COMPLETER //

```

Exercice 5 :

Implémentation de la fonction *atoi* pour un tableau d'entiers, puis de caractères

```

1
2 int* atoi_tab(char* chaine, int *taille){
3     int n = strlen(chaine);
4     int i = 0;
5
6     int cpt = 0;
7     int* tab = malloc(n*sizeof(int));
8     while(i<n){
9         if(chaine[i] != '\0'){
10             int j = i+1;
11             while(j<n && chaine[j] != '\0'){
12                 j+=1;
13             }
14             char chaine_i[j-i+1];
15             int l = 0;
16             int temp = i;
17             chaine_i[j-i] = 0;
18             while(l<(j-i)){
19                 chaine_i[l] = chaine[temp];
20                 l+=1;
21                 temp+=1;
22             }
23             tab[cpt] = atoi(chaine_i);
24             cpt++;
25             i = j;
26         }
27         else{
28             i+=1;
29         }
30     }
31     *taille=cpt;
32     return tab;
33 }
34
35 void affiche_tab(int* tab, int n){
36     int i = 0;
37     while(i<n){
38         printf("%d_", tab[i]);
39         i+=1;

```

```

40     }
41     printf("\n");
42 }
43
44 int main(int argc, char** argv){
45     int i = 1;
46     int taille;
47     while (i<argc){
48         int* tab = atoi_tab(argv[i], &taille);
49         affiche_tab(tab, taille);
50         free(tab);
51         i+=1;
52     }
53 }
54
55 // Q2 :
56 #include <ctype.h>
57
58
59 int* atoi_tab2(char* chaine, int *taille){
60     int n = strlen(chaine);
61     int i = 0;
62
63     int cpt = 0;
64     int* tab = malloc(n*sizeof(int));
65     while(i<n){
66         if(isdigit(chaine[i])!=0){
67             int j = i+1;
68             while(j<n && (isdigit(chaine[j])!=0)){
69                 j+=1;
70             }
71             char chaine_i[j-i+1];
72             int l = 0;
73             int temp = i;
74             chaine_i[j-i] = 0;
75             while(l<(j-i)){
76                 chaine_i[l] = chaine[temp];
77                 l+=1;
78                 temp+=1;
79             }
80             tab[cpt] = atoi(chaine_i);
81             cpt++;
82             i = j;
83         }
84         else{
85             i+=1;
86         }
87     }
88     *taille=cpt;
89     return tab;
90 }
91
92 int main(int argc, char** argv){
93     int i = 1;
94     int taille;
95     while (i<argc){
96         int* tab = atoi_tab2(argv[i], &taille);
97         affiche_tab(tab, taille);
98         free(tab);

```

```

99         i+=1;
100     }
101     Return EXIT_SUCCESS;
102 }

```

Exercice 6 :

Implémentation du *selection sort* pour des tableaux d'entiers et de caractères

```

1 // Q1 :
2
3 //Implementation du tri par selection
4
5 void selection_sort(int* tab, int n){
6     int i = 0;
7
8     int indx_ppvl = i;
9     int buff_ppl = tab[0];
10
11     while(i<n){
12         int j = i+1;
13         indx_ppvl = i;
14         buff_ppl = tab[i];
15         while(j<n){
16             if(tab[j] < buff_ppl){
17                 buff_ppl = tab[j];
18                 indx_ppvl = j;
19             }
20             j+=1;
21         }
22         int buff = tab[i];
23         tab[i] = buff_ppl;
24         tab[indx_ppvl] = buff;
25         i+=1;
26     }
27 }
28
29 // Test Q1 :
30
31 void affiche_tab(int* tab, int n){
32     int i = 0;
33     while(i<n){
34         printf("%d_", tab[i]);
35         i+=1;
36     }
37     printf("\n");
38 }
39
40
41 int main(int argc, char** argv){
42     int i = 1;
43     int* tab = malloc((argc-1)*sizeof(int));
44     while(i<argc){
45         int x = atoi(argv[i]);
46         tab[i-1] = x;
47         i+=1;
48     }

```

```

49     affiche_tab(tab, argc-1);
50     selection_sort(tab, argc-1);
51     affiche_tab(tab, argc-1);
52     free(tab);
53     return EXIT_SUCCESS;
54 }
55
56 // Q2 :
57
58 void lexico_sort(char** tab, int n){
59     int i = 0;
60
61     int indx_ppl = 0;
62     char* buff_ppl = tab[0];
63
64     while(i<n){
65         int j = i+1;
66         indx_ppl = i;
67         buff_ppl = tab[i];
68         while(j<n){
69             if(strcmp(buff_ppl, tab[j]) > 0){
70                 buff_ppl = tab[j];
71                 indx_ppl = j;
72             }
73             j++;
74         }
75         char* buff = tab[i];
76         tab[i] = buff_ppl;
77         tab[indx_ppl] = buff;
78         i++;
79     }
80 }
81
82 void affiche_tab_char(char* chaine){
83     int n = strlen(chaine);
84     int i = 0;
85     while(i<n){
86         printf("%c",chaine[i]);
87         i++;
88     }
89     printf("\n");
90 }
91
92 int main(int argc, char** argv){
93     lexico_sort((argv+1), argc-1); // Ici il faut decaler le pointeur pour eviter
                                     de prendre en compte ./exo6_exec dans le tri. Il faut alors diminuer la
                                     taille de argc.
94     int i = 1;
95     while(i<argc){
96         affiche_tab_char(argv[i]);
97         i++;
98     }
99     return EXIT_SUCCESS;
100 }

```